

Práctica II — Análisis Experimental de Algoritmos

DialSort vs. TieredSort

ST0245 - SI001 · Estructuras de Datos y Algoritmos · Universidad EAFIT
Docente: Alexander Narváez Berrío · Mayo 2026

Autores: Matias Zapata Rojas · Samuel Valencia Montoya

1. Objetivo

Implementar y comparar experimentalmente **DialSort** — un algoritmo de ordenamiento no comparativo basado en el principio de auto-indexación — contra **TieredSort**, una propuesta alternativa de dos niveles diseñada para reducir el footprint de caché en universos grandes. El análisis cubre complejidad algorítmica, tiempo real de ejecución y comportamiento ante distintas distribuciones de datos.

2. Descripción de los Algoritmos

2.1 DialSort

DialSort construye un histograma plano $H[0..U-1]$ en un único paso de ingesta sobre el arreglo de entrada. Cada clave k incrementa $H[k]$. En el segundo paso, un barrido lineal sobre el histograma emite cada valor k exactamente $H[k]$ veces, produciendo la salida ordenada. No requiere comparaciones ni prefix-sum.

Complejidad: $O(n + U)$ tiempo · $O(U)$ espacio · no comparativo

2.2 TieredSort (Propuesta del Estudiante)

TieredSort es un ordenamiento no comparativo de dos niveles. Divide el universo $[mn, mx]$ en $K = \text{ceil}(\sqrt{U})$ cubos gruesos de ancho igual ($\text{tier_width} = \text{ceil}(U/K)$). Dentro de cada cubo aplica un histograma local de tamaño tier_width .

Analogía: ordenar cartas por ciudad primero, luego por calle dentro de cada ciudad.

Paso 1 — Scatter: cada clave se asigna a su cubo mediante división entera: $\text{tier} = (k - mn) / \text{tier_width}$. $O(n)$. Paso 2 — Histograma fino: para cada cubo no vacío se construye un histograma local y se emiten los valores en orden. $O(n + U)$ total.

Complejidad: $O(n + U)$ tiempo · $O(n + \sqrt{U})$ espacio · no comparativo Ventaja clave: el working set activo por pasada es $O(\sqrt{U})$ en lugar de $O(U)$, lo que mejora la localidad de caché para universos grandes.

3. Comparación de Complejidad

Algoritmo	Tiempo	Espacio	Comparativo	Pasadas
std::sort	$O(n \log n)$	$O(\log n)$	Sí	—
DialSort	$O(n + U)$	$O(U)$	No	2
DialSort-Parallel	$O(n/p + U)$	$O(p \cdot U)$	No	2
TieredSort	$O(n + U)$	$O(n + \sqrt{U})$	No	3

Tabla 1. Comparación de complejidad algorítmica.

4. Diseño del Benchmark

El benchmark compara los cuatro algoritmos sobre las mismas entradas generadas con semilla fija (20260321) para garantizar reproducibilidad. Se mide el mejor tiempo de 7 ejecuciones, descartando 3 de calentamiento.

Parámetro	Valores
Tamaño de entrada (n)	10,000 · 100,000 · 1,000,000 · 10,000,000
Tamaño de universo (U)	256 · 1,024 · 65,536
Distribuciones	uniforme · sesgada · ordenada · inversa
Rondas de medición	mejor de 7 (3 warmup descartados)
Configuraciones totales	48

Tabla 2. Parámetros del benchmark.

5. Resultados Experimentales

5.1 Resumen General

Métrica	Valor
Configuraciones medidas	48
DialSort más rápido que TieredSort	48 / 48
TieredSort más rápido que DialSort	0 / 48
Ratio promedio DialSort / TieredSort	0.595x
Ratio mínimo (DialSort más ventajoso)	0.291x
Ratio máximo (DialSort menos ventajoso)	0.987x
Verificación de correctitud	PASSED (48/48)

Tabla 3. Resumen de resultados. Ratio < 1.0 indica que DialSort es más rápido.

5.2 Resultados Destacados — n = 10,000,000

Dist.	U	std::sort (ms)	DialSort (ms)	TieredSort (ms)	Speedup DS vs std
uniform	256	1,089.6	161.1	258.7	6.76x
skewed	256	1,073.8	169.5	313.7	6.34x
sorted	256	571.9	149.6	276.1	3.82x
uniform	1,024	1,112.8	154.2	269.3	7.22x
uniform	65,536	1,614.2	182.5	340.1	8.84x

Tabla 4. Tiempos de ejecución seleccionados para n = 10,000,000.

6. Análisis y Conclusiones

DialSort domina en todos los escenarios medidos. Su histograma plano, aunque de tamaño O(U), cabe en caché L1/L2 para U = 256 y U = 1,024, lo que lo hace extremadamente eficiente. Para U = 65,536 (256 KB de histograma) el acceso comienza a presionar L2/L3, pero DialSort sigue siendo más rápido porque realiza solo 2

pasadas lineales sobre el arreglo.

TieredSort reduce el footprint de caché activo por pasada de $O(U)$ a $O(\sqrt{U})$, lo cual es la ventaja teórica. Sin embargo, la tercera pasada adicional y el overhead de gestionar K cubos de vectores dinámicos superan el beneficio de caché en los rangos de U probados. TieredSort sería más competitivo con U en el orden de millones, donde el histograma plano de DialSort ya no cabe en LLC.

DialSort-Parallel supera a la versión secuencial para $n \geq 1,000,000$ gracias a la ingesta paralela del histograma con 8 hilos. Para n pequeño, el overhead de creación de hilos lo hace más lento que la versión secuencial.

Ambos algoritmos son correctos en el 100% de las configuraciones, verificado mediante la función `check_sorted()` tras cada ejecución.

Repositorio: <https://github.com/matias910/Practica-Final---Estructura-de-Datos-y-Algoritmos>